Manual Técnico de Software

Modelo de Gestión Curricular Institucional

Autores:
HÉCTOR RODRÍGUEZ LOYA
ISAAC GONZÁLEZ AGUILERA
LUIS ARTURO HERNANDEZ CASTILLO
FERNANDO GRIJALVA FERNÁNDEZ
PABLO GAEL TORRES

April 28, 2026

Disclaimer

El presente documento ha sido elaborado para la materia Ingenieria de Software II clave: 8CC2 para la carrera Ingenieria en Ciencias de la Computacion de la Universidad Autonoma de Chihuahua

Copyright

© [2026] [Developers del Norte]

El contenido, diseño, código fuente, logotipos y demás elementos de este sitio son propiedad de Developers del Norte y están protegidos por las leyes de propiedad intelectual. Queda prohibida su reproducción total o parcial sin autorización expresa.

Contact

a363325@uach.mx
a368048@uach.mx
a368073@uach.mx
a367634@uach.mx
a367685@uach.mx

Repositorio

`https://github.com/ektorex/Modelo-de-Gesti-n-Curricular-Institucional.git`

Changelog

v1.0	2026-28-04	Primera Edición

Table of Contents

1 Introducción **4**
1.1 OBJETIVO DE ESTE MANUAL 4
2 ASPECTOS TÉCNICOS **5**
2.1 HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO 5
2.2 Diseño de base de datos 6
2.3 Diseño de la API 7
3 Manual de Uso **10**
4 Instalación **17**
5 Conclusiones **18**
5.1 Alcances y limitaciones 18
5.2 Trabajo futuro. 18
5.3 Recomendaciones 18

1 Introducción

El Modelo de Gestión Curricular Institucional MGCI fue construido con el proposito de reemplazar el software antiguo que ya ha quedado rezagado, para alcanzar esta meta se aplicaron técnicas de re-ingenieria de software vistas en el curso.

El MGCI está compuesto de forma que permita que la información contenida pueda ser agregada, editada y/o modificada por el personal encargado del departamento.

En cuanto a este manual se ha considerado incluir todos los aspectos técnicos necesarios para el manejo y control del software.

1.1 OBJETIVO DE ESTE MANUAL

El objetivo primordial de este manual es el explorar y justificar las deciciones técnicas asi como el stack utilizado y mostrar las ventajas que ofrece respecto a su antecesosr. Asi como guiar y ayudar al usuario a utilizar las herramientas que el software ofrece.

Stack: conjunto de lenguajes de programación, frameworks, bases de datos, herramientas de desarrollo y servidores utilizados para construir y ejecutar software

1.2 DIRIGIDO

El presente documento está dirigido a la comunidad académica y administrativa involucrada en la gestión de planes de estudio, particularmente a docentes, personal administrativo y responsables de la coordinación académica.

Asimismo, se orienta a estudiantes y desarrolladores que requieran comprender el funcionamiento, alcance y uso del sistema propuesto, tanto desde una perspectiva operativa como técnica.

2 ASPECTOS TÉCNICOS

2.1 HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO

Es ésta sección se procede a explicar las herramientas informáticas empleadas para el desarrollo.

2.1.1 GitHub

GitHub es una plataforma de desarrollo colaborativo de software para alojar proyectos utilizando el sistema de control de versiones Git.

2.1.2 Docker

Docker es una plataforma de código abierto que permite a los desarrolladores empaquetar aplicaciones y todas sus dependencias (código, bibliotecas, configuración) en un contenedor ligero y portable.

2.1.3 PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacionales (RDBMS) de código abierto y objeto-relacional. Es reconocido por su gran fiabilidad, robustez, alto rendimiento y cumplimiento riguroso de los estándares SQL.

2.1.4 FastAPI

FastAPI es un framework web moderno y de alto rendimiento para construir APIs con Python 3.7+ basado en estándares.

2.1.5 HTML + CSS + Javascript

HTML, CSS y JavaScript son los tres lenguajes fundamentales del desarrollo web front-end (lado del cliente).

2.1.6 Bash

Bash (Bourne-Again SHell) es un intérprete de comandos y un lenguaje de programación de guiones (scripting) utilizado para interactuar con sistemas operativos basados en Unix.

2.2 Diseño de base de datos

Nuestra propuesta para la nueva base de datos consiste de cuatro tablas, siendo las principales **materias** y **carreras**, estando estas relacionadas por la tabla **planes**, de modo tal que diferentes carreras puedan acceder a la misma materia pero dejando planes de estudio, si bien en relacion uno a uno con las carreras, independientes para poder ser modificados o eliminados sin afectar las carreras ni las materias.

Ademas las materias estan relacionadas entre si por medio de la tabla **prerequisitos** tal que estas puedan tener tantas materias previas como sea requerido, incluyendo ninguna.

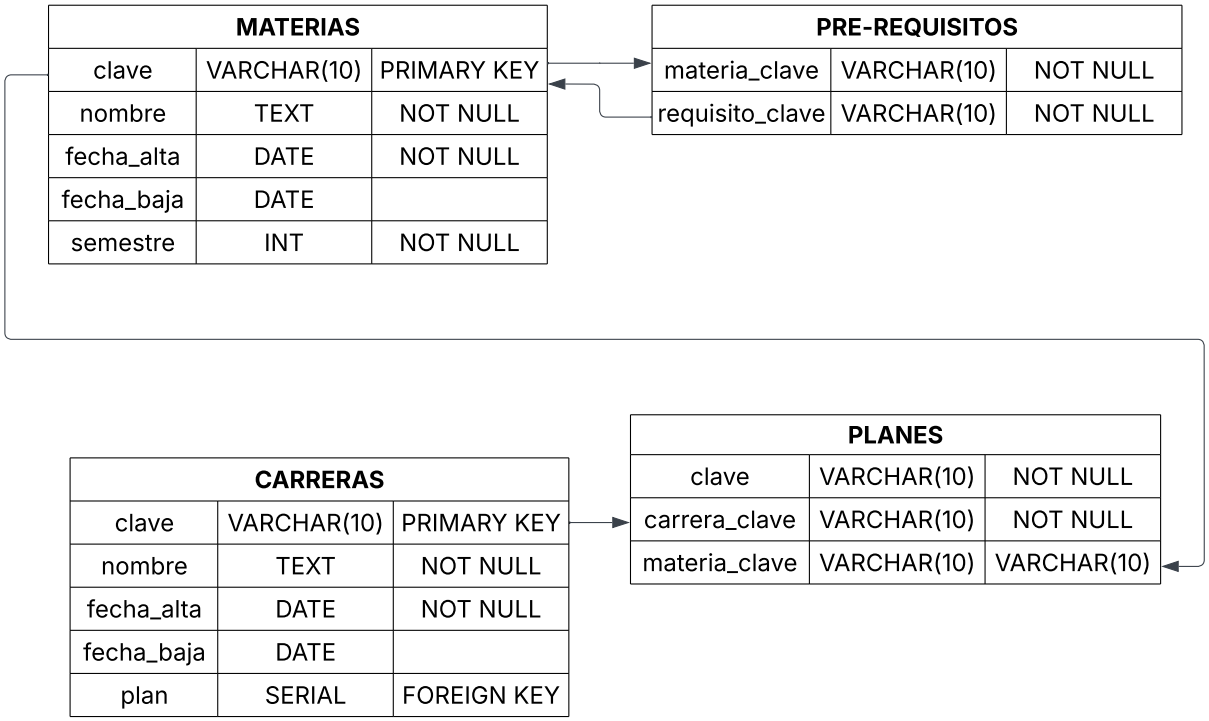

Figure 1: Diagrama del esquema de la base de datos propuesta

Se añadieron constrains tomando en cuenta las siguientes consideraciones:

- No se permite eliminar una materia que sea pre-requisito de otra.
- No se permite eliminar una materia que se encuentre en un plan educativo.
- No se permite eliminar un plan que este activo (referenciado por una carrera).

2.3 Diseño de la API

Se optó por separar la carga de las APIs en dos componentes diferenciados:

Por un lado, se implementó una API orientada exclusivamente a la visualización de información, compuesta únicamente por métodos GET, diseñada para poder ser expuesta de forma segura sin comprometer la integridad del sistema.

Por otro lado, se desarrolló una API destinada a la modificación de la base de datos, cuyos endpoints están restringidos a un entorno local, con el objetivo de mitigar riesgos de seguridad y controlar el acceso a operaciones sensibles.

2.3.1 Enpoints para visualizacion (GET)

/carreras

```
SELECT * FROM carreras
```

/materias

```
SELECT * FROM carreras
```

/planes/carrera

```
SELECT m.clave, m.nombre, m.semestre
FROM planes p
JOIN materias m ON p.materia_clave = m.clave
WHERE p.carrera_clave = %s
ORDER BY m.semestre
```

/materias/clave

```
cur.execute("""
    SELECT clave, nombre, semestre
    FROM materias
    WHERE clave = %s
""", (clave,))
materia = cur.fetchone()

cur.execute("""
    SELECT requisito_clave
    FROM prerequisitos
    WHERE materia_clave = %s
""", (clave,))
reqs = cur.fetchall()

cur.close()
conn.close()

return {
    "materia": materia,
    "requisitos": [r[0] for r in reqs]
}
```

2.3.2 Enpoints para admin (POST)

/materias

```
    cur.execute(
        "INSERT INTO materias (clave, nombre, fecha_alta,
            semestre) VALUES (%s, %s, %s, %s)",
        (m.clave, m.nombre, m.fecha_alta, m.semestre)
    )
```

/prerequisitos

```
    cur.execute(
        "INSERT INTO prerequisitos (materia_clave,
            requisito_clave) VALUES (%s, %s)",
        (p.materia_clave, p.requisito_clave)
    )
```

/planes

```
    cur.execute(
        "INSERT INTO planes (clave, carrera_clave, materia_clave)
             VALUES (%s, 'PEND', %s)",
        (p.clave, p.materia_clave)
    )
```

2.3.3 Enpoints para admin (DELETE)

/materias/clave

```
    try:
        cur.execute("DELETE FROM materias WHERE clave = %s", (clave,))
        conn.commit()
    except Exception as e:
        conn.rollback()
        return {"error": str(e)}
    finally:
        cur.close()
        conn.close()
```

/carreras/clave

```
    try:
        # opcional: liberar planes a PEND
        cur.execute("""
            UPDATE planes SET carrera_clave = 'PEND'
            WHERE carrera_clave = %s
        """, (clave,))

        cur.execute("DELETE FROM carreras WHERE clave = %s", (clave,))
        conn.commit()

    except Exception as e:
        conn.rollback()
        return {"error": str(e)}
    finally:
        cur.close()
        conn.close()
```

/planes/clave

```
    try:
        # evita borrar si ya est  asignado a carrera
        cur.execute("""
            SELECT 1 FROM planes
            WHERE clave = %s AND carrera_clave != 'PEND'
            LIMIT 1
        """, (clave,))

        if cur.fetchone():
            return {"error": "El plan ya est  asignado a una carrera"}

        cur.execute("DELETE FROM planes WHERE clave = %s", (clave,))
        conn.commit()

    except Exception as e:
        conn.rollback()
        return {"error": str(e)}
    finally:
        cur.close()
        conn.close()
```

3 Manual de Uso

3.0.1 Panel administrativo

Para crear una nueva materia debe llenar los campos solocitados: clave, nombre y semestre. de requerir pre-requisitos puede seleccionar multiples opciones utilizando la tecla ctl al hacer click. Una vez llenados los campos hacer click en el boton Registrar Materia. Vea Figura 2.

Figure 2: Campo de creacion de materias

Para crear una carrera basta con llenar los campos solocitados: clave, nombre y seleccionar uno de los planes de estudio disponibles. Vea Figura 3.

NOTA: Solo estaran disponibles planes no asignados a una carrera

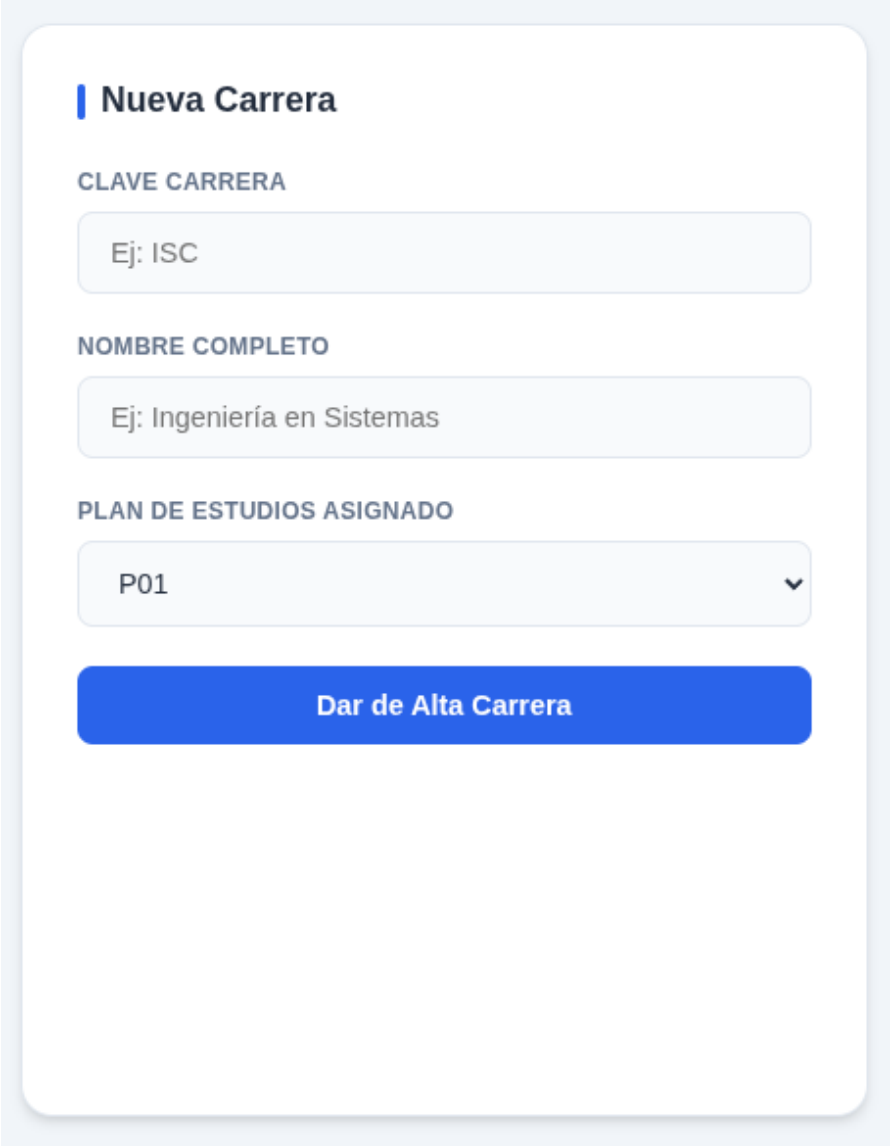

Figure 3: Campo de creacion de carreras.

Para crear un nuevo plan de estudio basta con ingresar su clave y seleccionar las materias que contendra. Vea Figura 4.

NOTA: Para seleccionar multiples materias oprima la tecla ctl al hacer click

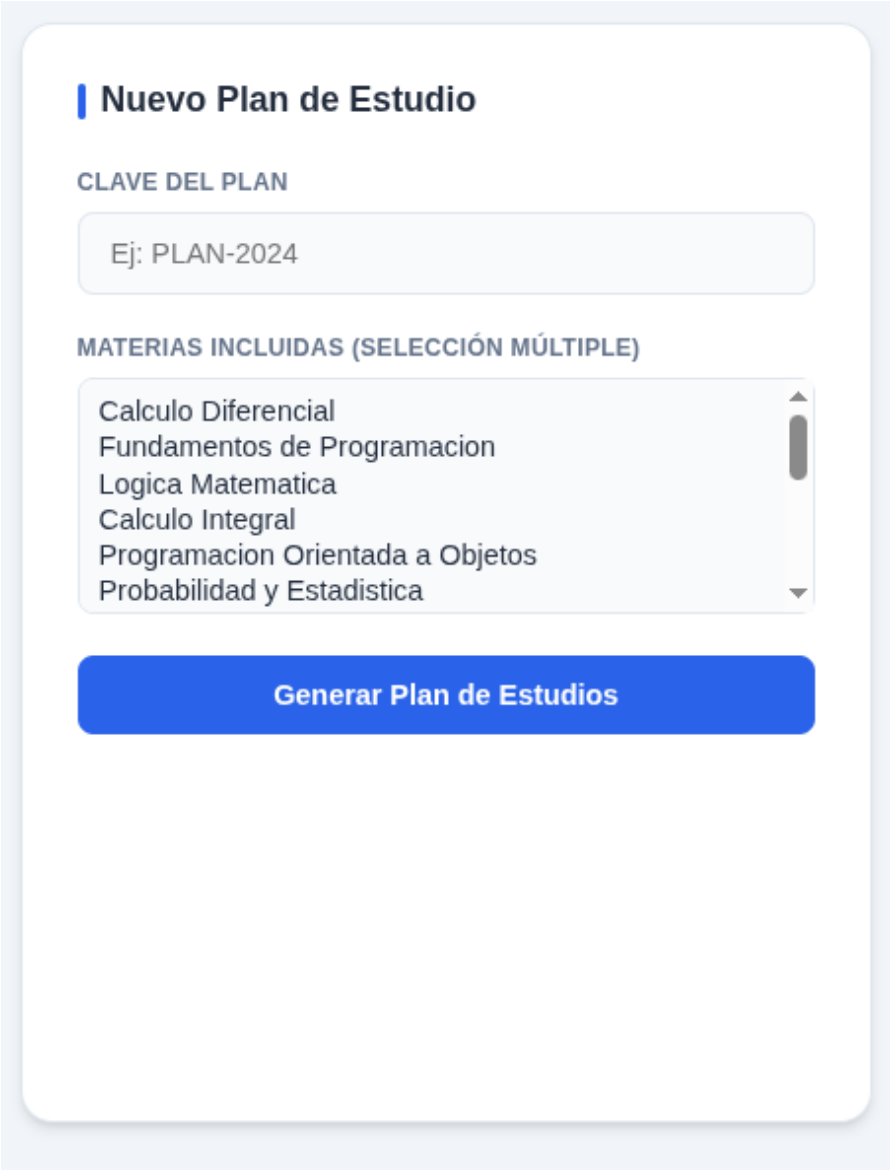

Figure 4: Campo de creacion de plane de estudio.

Para eliminar ya sea una matera, plan o carrera, basta con referenciar su clave y presionar el boton Eliminar. Vea Figura 5.

NOTA: Es posible que no se efectue la eliminacion, esto se debe a que el objeto a eliminar depende de algun otro el orden correcto es 1. Carrera, 2. Plan, 3. Materia (si no esta en algun otro plan o es requisito de otra materia)

Figure 5: Campo de eliminacion

3.0.2 Panel de visualizacion

Basta con hacer click la carrera de interes para mostrar su plan de estudio. Ver Figura 6.

Figure 6: Campo de carreras

Basta con hacer click en la materia de interes para mostrar sus detalles. Vea Figura 7.

Figure 7: Campo de planes de estudio

Aqui se muestran los detalles de la materia seleccionada. Vea Figura 8.

Figure 8: Detalles de la materia

3.0.3 Estructura del proyecto

El proyecto está compuesto por doce archivos; sin embargo, únicamente es necesario ejecutar aquellos ubicados en el directorio principal: init, killer y server. Para una mejor referencia de la estructura del proyecto, consulte la Figura 9.

```
.
├── api
│   ├── admin.py
│   ├── db.py
│   └── main.py
├── dataBase
│   ├── docker-compose.yaml
│   ├── init.sql
│   └── seed.sql
├── frontend
│   ├── admin
│   │   └── index.html
│   └── user
│       └── index.html
├── init
├── killer
└── server
```

Figure 9: Arbol de la estructura del proyecto

4 Instalación

Para facilitar la instalación del sistema, se incluye un script en Bash que automatiza la configuración inicial y la puesta en marcha de sus componentes. Este script se encarga de inicializar la base de datos y de levantar, en un entorno local, tanto las interfaces web como sus respectivas API.

Existen diversas alternativas para exponer la interfaz de usuario en la red; sin embargo, dichas configuraciones no forman parte del alcance de este proyecto.

Para iniciar el sistema por primera vez, ejecute el script init. Para detener los servicios en ejecución, utilice killer. Posteriormente, puede emplear el script server para volver a levantar las aplicaciones.

NOTA: para ejecutar un script escriba ./nombre-del-script, ej: ./init , ./killer ./server

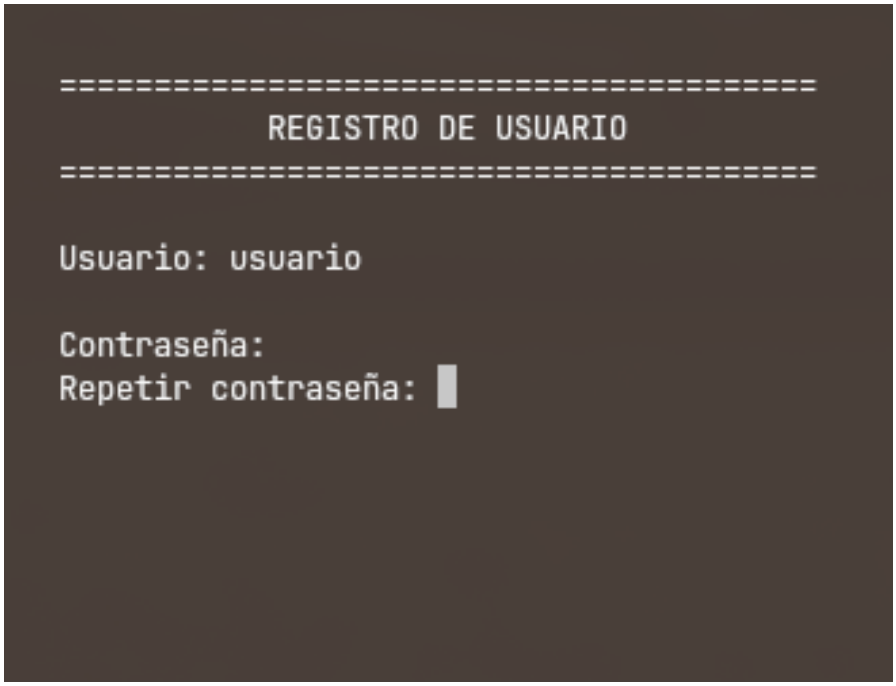

Figure 10: Unicamente se pide crear un usuario y contraseña para la base de datos

5 Conclusiones

El sistema desarrollado cumple con los objetivos planteados, proporcionando una solución estructurada, segura y eficiente para la gestión de información académica. La arquitectura implementada, basada en la separación de responsabilidades entre servicios de consulta y modificación, permite mejorar la seguridad, el control de acceso y la mantenibilidad del sistema.

5.1 Alcances y limitaciones

La solución está orientada a entornos locales controlados, donde se garantiza la integridad de las operaciones críticas. No se contemplan, en esta versión, mecanismos avanzados de autenticación ni despliegues en entornos distribuidos.

5.2 Trabajo futuro

Se plantea como trabajo futuro la incorporación de mecanismos de autenticación, control de roles, así como la adaptación del sistema para su despliegue en entornos de red más amplios sin comprometer la seguridad.

5.3 Recomendaciones

Se recomienda realizar respaldos periódicos de la base de datos, mantener actualizadas las dependencias del sistema y restringir el acceso a los endpoints de modificación únicamente a entornos seguros.